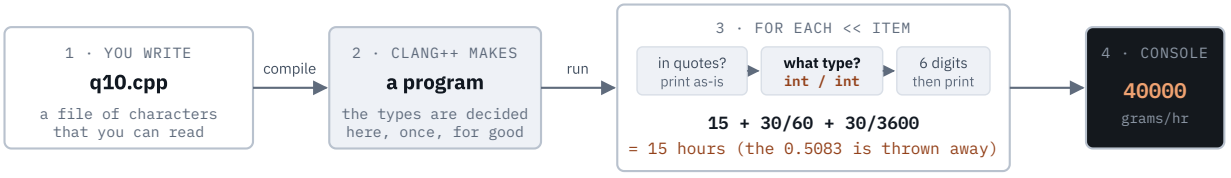
GEN 241 · ELECTRICAL POWER AND MACHINES · SHEET 1 · LAB WEEK 1

A program prints what you wrote, not what you meant

One stall sells `600 kg in 15 h 30 min 30 s`. Sheet 1 question 10 asks for the average sale in grams per hour. The obvious program answers **40000**. The right answer is **38688.9**. Everything in this guide is about those missing 1311 grams, because the same mistake is hiding in question 7 and in half the programs you will write this term.

How the time is written in the source



Every box except one is identical in both versions: same file, same compiler, same operator, same 600000 grams.
Four characters inside box 3 move the console from 40000 to 38688.9.

Chapters 1-2 cover boxes 1, 2 and 4. Chapters 3-5 live inside box 3. Chapter 6 is the last step before the console.

WHAT CHANGES

The division inside the brackets is whole-number division, so 30/60 and 30/3600 are both 0.

WHAT STAYS THE SAME

The file, the compiler, the 600000 grams, the operator and the printing rule.

THE QUESTION IT ANSWERS

What does C++ do when both sides of a slash are whole numbers?

The compiler decides the type once, and everything downstream obeys it. Nothing about the stall, the division or the printing changes between the two settings. Only how the two numbers beside the slash were typed.

Read it in order once

The worksheet

Chapters follow the arrows in the figure above, left to right. Every widget on the page computes its numbers while you watch: nothing here is a screenshot.

One `.cpp` file holding every step, with the exercises left blank. Compile it before you start and again when you finish.

Before the lab

You need a working compiler and about 90 minutes. Chapter 1 sets up both VS Code and Dev-C++, so bring whichever machine you have.

CHAPTER 1

Getting something to run

After this chapter you can turn a file of text into a running program, on your own machine and on the lab machines, and you can tell a compile failure apart from a wrong answer.

Write › **Compile** › Evaluate › Print

1.1

Six lines, and a job for each one

This is the whole of Example 1 from the lecture. Nothing in it is decoration: take any line away and either the program stops compiling or it stops doing anything.

live-coding/01-hello.cpp

```cpp
#include <iostream>
using namespace std;

int main()
{
    // Display Welcome to C++ to the console
    cout << "Welcome to C++!" << endl;

    return 0;
}
```

OUTPUT

```
Welcome to C++!
```

1. `#include <iostream>` pastes the input/output library into your file before compiling. `cout` and `endl` are defined in there. Without this line the compiler has never heard of either name.

2. `using namespace std;` lets you write `cout` instead of `std::cout`. The library keeps its names in a namespace called `std` so they cannot collide with yours; this line says "look in there too".
4. `int main()` is where execution starts. Every C++ program has exactly one. The `int` says main hands a whole number back when it finishes.
5. The braces `{` and `}` mark the beginning and end of main's body. Everything between them runs in order, top to bottom.
6. A line comment. Everything after `//` is for humans; the compiler skips it. For several lines, wrap them in `/*` and `*/`.
7. The one line that does something. Note the semicolon: it ends the statement. Most first-week compile errors are a missing one.
9. `return 0;` hands 0 back to whoever started the program, which by convention means "finished, nothing went wrong".

KEY IDEA

Lines 1, 2, 4, 5 and 9 are scaffolding you will type a hundred times this term. Line 7 is the program. When you read someone else's code, find the lines that are not scaffolding first.

1.2

Compiling and running are two separate things

Your `.cpp` file is text. The processor cannot run text. A compiler reads the whole file and produces a second file, the program, which the processor can run. Two steps, two kinds of failure.

A **compile error** means no program was produced, so there is nothing to run. The message names a file and a line number: go there first, then look at the line *above* it, because a missing semicolon is usually reported on the following line. A **wrong answer** means compiling worked perfectly and your instructions were faithfully carried out - they were just not the instructions you wanted. Chapters 3 to 6 are entirely about the second kind.



A clean compile says nothing about whether the answer is right. The compiler checks that your sentences are grammatical C++, never that they mean what you intended.

بالمصري

الكمبايلر بيترجم الملف بتاعك لبرنامج. دي خطوة، وتشغيل البرنامج دي خطوة تانية خالص.

لو الترجمة وقعت، يبقى مفيش برنامج أصلاً علشان يشتغل. ولو الترجمة نجحت، ده معناه إن كلامك مظبوط لغويًا بس - مش معناه إن النتيجة صح.

1.3

Two compilers, one language

The lab machines run Dev-C++ on Windows. This guide was written and checked on a Mac using VS Code with Apple's `clang++`. Every program here compiles unchanged on both, because nothing in Sheet 1 goes near the parts of C++ where compilers differ. What differs is only the two clicks that get you from file to output.

IN THE LAB · DEV-C++

File, then F11

`File → New → Source File`, type the program, save it as something ending in `.cpp`, then press `F11` to compile and run in one go. `F9` compiles without running.

If the black window appears and vanishes instantly, the program finished before you could read it. Dev-C++ usually holds it open with *Press any key to continue*; if yours does not, add `system("pause");` just before `return 0;` and `#include <cstdlib>` at the top.

ON A MAC · VS CODE

Open the file, then Cmd+Shift+B

Open the course folder in VS Code, click into any `.cpp` file, and press `⌘⇧B`. The task compiles the file you are looking at and runs it in the terminal panel underneath.

The output appears under a line reading `----- output -----`. Anything printed above that line is the compiler talking to you, not your program.

WHAT COULD FOOL YOU

- **Smart quotes.** If you draft your code in Word or paste it from a PDF, `"` often arrives as `“` and `”`. The compiler does not recognise those. Type your quotes in the editor.
- **The file extension.** Saving as `hello.txt` or `hello` gets you a file nothing will compile. It must end in `.cpp`.
- **A missing semicolon** is usually reported on the line *after* the one you need to fix, because that is where the compiler finally gave up.
- **Reading the last error first.** One mistake often produces a cascade. Fix the topmost message, recompile, and watch most of the rest disappear.
- **Editing while the old program runs.** Fixing the file does not change the program you already built. Compile again, every time.

CHECK YOURSELF

You compile, get no errors at all, run the program, and the number it prints is wrong. What have you learned?

A The compiler must have a bug

B Your instructions were followed exactly, and they were the wrong instructions

C You probably forgot a semicolon

Back to the sheet: with this much you can already finish **Q1** and **Q2**. Open `week-01-first-programs/sheet-01/q01-course-header.cpp` and run it before reading on.

CHAPTER 2

cout sends items, one after another

After this chapter you can look at a cout statement and say exactly what appears on the console, including every space and every line break, before you run it.

Write › Compile › Evaluate › **Print**

2.1

Each `<<` hands over one more item

Read `cout << a << b << c;` as a conveyor belt running left to right. The console has a cursor; each item is written starting wherever the previous item stopped. Nothing is inserted between items - not a space, not a line break. If you want a gap, you have to send one.

That is the entire behaviour, and it explains every layout surprise in Sheet 1. Step through the stall's output line below and watch the cursor move.

one statement, four items

```
cout << "average = " << average << " grams/hr" << endl;
```

0 of 4 items sent

CONSOLE

Nothing has been sent yet. The cursor is at column one and the console is empty.

Nothing appears between two items unless you put it there. The space before `grams/hr` exists only because it was typed inside the quotes; delete it and the output reads `38688.9grams/hr`.

KEY IDEA

An item in quotes is printed character for character. An item not in quotes is worked out first, and its answer is printed. Everything in chapters 3 to 6 is about that second kind.

2.2

Spaces inside quotes are data (Q3)

Question 3 asks for two triangles side by side. There is no drawing instruction in C++ and none is needed: each line of the picture is one string, and the spaces inside it are as real as the asterisks. Every line below is exactly 20 characters wide - nine for the left triangle, two for the gap, nine for the right one.

Spaces

Q03 OUTPUT · 20 COLUMNS

```
    *       *********
   ***       *******
  *****       *****
 *******       ***
*********       *
```

This is what the console shows. Switch to the other view to see where the 51 space characters actually are.

The picture is made of spaces as much as of asterisks. The left triangle grows by one asterisk on each side per line while the right one shrinks, and the two-column gap between them never moves.

sheet-01/q03-triangle-pattern.cpp · the five lines

```
cout << "    *      *********" << endl;
cout << "   ***      ******* " << endl;
cout << "  *****      *****  " << endl;
cout << " *******      ***   " << endl;
cout << "*********      *    " << endl;
```

بالمصري

المسافات اللي جوه علامتي التنصيص دي جزء من الكلام نفسه، مش شكل ولا تنسيق.

لو شِلت مسافة واحدة بالغلط، الرسمة كلها هتتزحلق. عدّ المسافات بالظبط قبل ما تجري البرنامج.

2.3

Characters you cannot type directly

A double quote would end the string early, and there is no key for "move to the next column". C++ writes these as a backslash plus a letter. The pair counts as **one** character.

You type	What lands on the console	Where Sheet 1 needs it
`\n`	End this line, start a new one	An alternative to `endl` anywhere
`\t`	Jump to the next tab stop	Q4, to line the table columns up
`\"`	A double quote	Whenever you quote something
`\\`	One backslash	Printing a Windows file path

`\t` is not "a few spaces". It moves the cursor to the next tab stop, and the console decides where those are, usually every 8 columns. That is why Q4's table lines up without you counting anything, and also why it can look wrong if one entry is long enough to overshoot a stop.

sheet-01/q04-table.cpp

```cpp
cout << "P\tP*5\tP*10" << endl;
cout << "5\t25\t50" << endl;
cout << "10\t50\t100" << endl;
cout << "25\t125\t250" << endl;
cout << "50\t250\t500" << endl;
```

OUTPUT

```
P       P*5     P*10
5       25      50
10      50      100
```

```
25      125     250
50      250     500
```

✓ CHECKED AGAINST THE LECTURE

The reference solution for Q4 heads these columns `p^2` and `p^3`, but the numbers underneath are P×5 and P×10: the second row is 10, 50, 100, and 10^3 is 1000, not 100. The sheet asks for `P`, `P*5`, `P*10`, and that is what the file above prints. Your tutor knows about this one.

WHAT COULD FOOL YOU

- **Expecting a space between items.** `cout << "a" << "b";` prints `ab`. There is no separator.
- **Expecting a new line at the end.** Without `endl` or `\n` the next program output continues on the same line, and in some terminals your last line is not shown at all until something else prints.
- **Counting `\t` as a fixed number of spaces.** It is one character that jumps to a column, so the same `\t` can produce seven spaces on one row and two on the next.
- **Trailing spaces looking harmless.** In Q3 the spaces after the final asterisk are what keep all five strings the same length. An editor set to trim trailing whitespace on save will quietly eat them.
- **Using `\` instead of `/` in a division.** They are different characters; only `/` divides.

CHECK YOURSELF

What exactly does `cout << "GEN" << 241 << endl << "lab";` put on the console?

A GEN 241 then lab on the next line

B GEN241 then lab on the next line

C GEN241lab all on one line

Back to the sheet: **Q1, Q2, Q3 and Q4** need nothing beyond this chapter. Q3 is the one to do slowly, with the spaces counted.

CHAPTER 3

Let the program do the sums

After this chapter you can hand an arithmetic expression to cout and say, in order, which operation it performs first, second and last.

Write › Compile › **Evaluate** › Print

3.1

One pair of quotes changes everything

These two lines differ by two characters, and they do entirely different jobs. The first sends eleven characters to the console. The second performs a multiplication and an addition and sends the one number that comes back.

the difference in one screen

```
1  cout << "10.5 + 2 * 3" << endl;
2  cout <<  10.5 + 2 * 3  << endl;
```

OUTPUT

```
10.5 + 2 * 3
16.5
```

KEY IDEA

Working the answer out on your calculator and typing `cout << "16.5";` is not a program. It is a picture of an answer. Give C++ the question and it stays correct when the numbers change.

3.2

Order of operations, and why Q5 needs its brackets

C++ follows the ordinary rules: `*` and `/` before `+` and `-`, left to right among equals, and anything in brackets first. Question 5 asks for a fraction, and a fraction is really a pair of brackets: everything on top is worked out, everything underneath is worked out, and only then does the division happen.

Step through it below. Each click performs exactly one operation, innermost brackets first.

```
(1.2*0.1 + 3.3*0.3) / (0.09 + 0.001)
```

Nothing has been worked out yet. The brackets are done first, and inside each bracket the multiplications come before the additions.

WITH THE BRACKETS · Q5 AS ASKED

12.1978

`(1.2*0.1 + 3.3*0.3) / (0.09 + 0.001)`

SAME CHARACTERS, BRACKETS DROPPED

11.121

`1.2*0.1 + 3.3*0.3 / 0.09 + 0.001`

Different question. Without the brackets, only `3.3*0.3` gets divided by `0.09`, and `0.001` is added at the end instead of forming part of the denominator.

A horizontal fraction bar is two pairs of brackets that you have to type. Both values above are computed by this page from the expressions printed under them.

3.3

There is no power operator

Questions 8 and 9 both need a side squared. C++ has no `^` for powers - the character exists but means something unrelated to arithmetic, so `9.2^2` will not even compile here. For a square, multiply the number by itself.

sheet-01/q08-equilateral-triangle.cpp

```cpp
cout << "Area      = " << 1.732 * (9.2*9.2) / 4 << endl;
cout << "Perimeter = " << 3 * 9.2 << endl;
```

OUTPUT

```
Area      = 36.6491
Perimeter = 27.6
```

1. The brackets around `9.2*9.2` are not strictly required, since `*` and `/` run left to right anyway. They are there so a reader can see the squared side at a glance. Write brackets for the reader, not only for the compiler.
2. Dividing by `4` is safe here even though 4 is a whole number, because `1.732 * 84.64` is already a decimal. Chapter 4 is about the case where it is not.

WHAT COULD FOOL YOU

- **`a/b*c` is not `a/(b*c)`.** Same precedence means strictly left to right, so it is `(a/b)*c`. Sheet 1 does not punish this, but Sheet 2 will.
- **Typing the answer instead of the question.** `cout << 12.1978;` scores nothing for Q5. The marker is looking for the expression.
- **Leaving out the multiply sign.** `2(3+1)` is mathematics, not C++. You must write `2*(3+1)`.
- **Unbalanced brackets.** Count the opening and closing ones. The compiler's message for this is usually unhelpful and points at the wrong line.

CHECK YOURSELF

You need to print the average of 4 and 10. Which line does it?

A `cout << 4 + 10 / 2;`

B `cout << (4 + 10) / 2;`

C `cout << (4 + 10) ^ 2;`

Back to the sheet: **Q5, Q6, Q8 and Q9** are all reachable now. Q6 is worth noticing - the ten `+` signs are additions, and cout receives the single number 100.

CHAPTER 4

The 1311 grams: int and double

After this chapter you can look at any division in a program and say whether C++ will do real arithmetic or throw the fraction away, before you run it.

Write › Compile › **Evaluate** › Print

4.1

The stall, written the obvious way

Question 10: a stall sells 600 kg in 15 hours, 30 minutes and 30 seconds; print the average sale in grams per hour. Two conversions, both of them things you would do without thinking on paper. 600 kg is 600000 g. 30 minutes is 30/60 of an hour and 30 seconds is 30/3600 of an hour. Written straight out:

live-coding/05-the-stall-wrong.cpp

```
cout << "average = " << 600e3 / (15 + 30/60 + 30/3600) << " grams/hr" << endl;
```

OUTPUT

```
average = 40000 grams/hr
```

On paper the answer is 38688.9. The program says 40000, it compiled without a single warning, and it will say 40000 every time you run it on every machine in the lab. Before reading on, decide where you think the 1311 grams went.

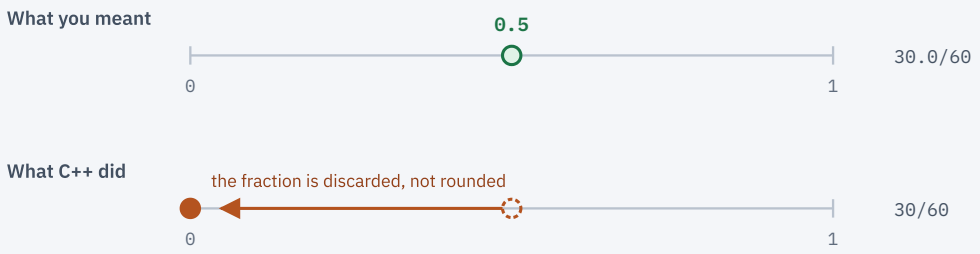
4.2

`30/60` is zero

C++ picks the kind of arithmetic from the operands, not from what you are trying to achieve. `30` and `60` are both whole numbers - the type is called `int` - so the division is a whole-number division, and a whole-number division produces a whole number. 30

divided by 60 is 0 remainder 30, so the answer is **0**. The remainder is not rounded. It is discarded.

The same happens to `30/3600`. The denominator the program actually used was `15 + 0 + 0`, a flat 15 hours, and 600000 divided by 15 really is exactly 40000. The program was right about the question it was given.



Rounding 0.5 would have given 1, which is also wrong. Integer division always moves toward zero.

Integer division truncates; it does not round. That is why the error is always a loss and never a wobble, and why it compounds: two truncated terms cost the stall 0.5083 of an hour.

KEY IDEA

If **both** sides of a `/` are whole numbers, C++ does whole-number division and throws the remainder away. If **either** side has a decimal point, it does real division. The fix is to give one side a decimal point: `30.0/60`.

بالمصري

القسمة بين رقمين صحيحين بترجع رقم صحيح. يعني `30/60` مش نص، دي `0`. والكسر مش بيتقرّب، بيتشال خالص.

علشان تاخد النص، لازم واحد من الاتنين يبقى فيه علامة عشرية: `30.0/60`. ده كل الفرق.

4.3

Try it on anything

Type an expression and this page will work it out twice: once the way a calculator would, and once following C++'s rules. When the two panels disagree, the reason is always the same and it is named underneath.

Expression, written exactly as you would in `cout << …`

`15 + 30/60 + 30/3600`

`30/60` `30.0/60` `the stall's hours` `the whole of Q10` `1/3`

`Q7, whole numbers` `9/2` `5/2*2` `4 + 10/2`

ON A CALCULATOR

15.5083

Every division done for real.

WHAT C++ PRINTS

15

Every operand was a whole number, so the result is an int and prints with no decimal part.

They differ. C++ reached `30/60`, found two whole numbers, and answered `0` instead of `0.5`, and it did the same thing 1 more time. Give one operand a decimal point to fix it.

Both columns are computed by this page as you type, the right one following C++'s rule that int divided by int stays int. Nothing here is a stored answer.

4.4

The same bug, wearing a different hat (Q7)

Question 7 approximates π with an alternating series and then asks you to discuss why writing `1` instead of `1.0` changes the result. It is the stall's bug again. In `1/3`, both operands are whole numbers, so the term is 0. Every term after the first collapses, the bracket is left holding a bare 1, and the program confidently prints **4**.

sheet-01/q07-pi-series.cpp

```cpp
cout << "Pi (using 1.0, with decimal point)    = "
     << 4.0 * (1.0 - 1.0/3 + 1.0/5 - 1.0/7 + 1.0/9 - 1.0/11 + 1.0/13 - 1.0/15)
     << endl;
cout << "Pi (using 1, without decimal point)   = "
     << 4 * (1 - 1/3 + 1/5 - 1/7 + 1/9 - 1/11 + 1/13 - 1/15)
     << endl;
```

OUTPUT

```
Pi (using 1.0, with decimal point)    = 3.01707
Pi (using 1, without decimal point)   = 4
```

How the numerators are written



Each term is smaller than the last and its sign flips, so the total crosses π on every row.

With 1 term the program prints 4, which is 0.8584 away from π and on the high side. Add another term and the total will cross to the other side of π.

The partial sums land either side of π in turn, which is why two students with different numbers of terms can both be right. Every dot is computed by this page from the series itself.

✓ CHECKED BY COMPILING BOTH

The sheet's formula has eight terms, ending at `-1/15`, and gives **3.01707**. The lecture screenshot stops at `+1/13`, seven terms, and gives **3.28374**. Neither is a mistake in the other's arithmetic: π is 3.14159, the seventh partial sum sits 0.1421 above it and the eighth sits 0.1245 below. Both files are in the course folder, and both were compiled to produce these numbers.

WHAT COULD FOOL YOU

- **Assuming the compiler will warn you.** It will not. `30/60` is a perfectly legal, extremely common thing to write, and `-Wall -Wextra` says nothing about it.
- **Thinking it rounds.** `9/2` is 4, not 5, and `1/3` is 0, not 0.333. The fraction is dropped, always toward zero.
- **Fixing only the first term.** In `1.0 - 1/3` the subtraction is real, but `1/3` was already 0 before the subtraction ever happened. Each division is judged on its own two operands.
- **Trusting a suspiciously round answer.** 40000 and 4 are both exactly the shape that integer division produces. A number that lands on a round figure when you expected a messy one deserves thirty seconds of suspicion.

- **Writing `30/60.0` versus `30.0/60`.** Both work. Only one operand needs the decimal point, and which one does not matter.

CHECK YOURSELF

In the stall's program, why does writing `30.0/60` fix an error in a term you did not touch, `30/3600`?

A It doesn't - but the error in that term is too small to matter

B It doesn't fix it - `30/3600` is still 0, and that term must be changed too

C Once one term is a double, the whole expression becomes real arithmetic

Back to the sheet: **Q7, Q10 and Q11** all turn on this chapter. Open the playground above and paste your own denominator into it before you submit.

CHAPTER 5

Naming the parts

After this chapter you can turn a one-line expression into a program whose quantities have names and units, and you can say what the declared type promises about each one.

Write › Compile › **Evaluate** › Print

5.1

Declare, assign, use

The stall's answer does not change when you give its parts names. What changes is that the units become visible, each number has one place to be edited, and the line that does the arithmetic finally reads like the sentence in the question.

live-coding/07-variables.cpp · the body of main

```
1 double sales, Time, average;
2
3 sales   = 600e3;                        // grams
4 Time    = 15 + 30.0/60 + 30.0/3600;     // hours
5 average = sales / Time;                 // grams per hour
6
7 cout << "average = " << average << " grams/hr" << endl;
```

(1) A **declaration**. It names three quantities and promises each will hold a number that may have a decimal part. Nothing has a value yet. Three names share one `double` here, separated by commas, which is what the lecture's solutions do.

(3) An **assignment**. The `=` is not an equation, it is an instruction: work out the right-hand side and put the result into `sales`. `600e3` means 600×10^3, so 600000 - a convenient way to write the kilogram-to-gram conversion without counting zeros.

(4) The conversion from chapter 4, now with a unit written beside it. Both `.0`s are load-bearing.

5 A name can be used on the right of `=` as soon as it has a value. Here `sales` and `Time` both do, so `average` gets 38688.9.

7 `average` is not in quotes, so cout prints its value. Put it in quotes and the console would read `average = average grams/hr`.

5.2

The type is a promise about what fits

`int` holds whole numbers only. `double` holds numbers with a decimal part. Choosing `int` for something that has a fraction does not produce an error, a warning, or a rounded value - it produces a silently truncated one, in exactly the way chapter 4 described. And it can bite in two different places. Switch the declaration below and watch the answer, not the code.

How the quantities are declared

the body of main, with only the declarations changing

```
double sales, Time;
double average;

sales   = 600e3;                        // grams
Time    = 15 + 30.0/60 + 30.0/3600;     // hours
average = sales / Time;
cout << "average = " << average << " grams/hr" << endl;
```

OUTPUT

```
average = 38688.9 grams/hr
```

Everything that can have a fraction is held in a double, so the stall's real rate survives all the way to the console.

Truncation happens wherever a value with a fraction meets a place that cannot hold one, not only at divisions. The same wrong answer from chapter 4 can be produced without writing a single whole-number division.

KEY IDEA

For every quantity in this course, ask one question: can it have a fraction? If yes, it is a `double`. Money, times, rates, averages, measurements and populations-that-grow-by-156.042-per-hour are all yes.

5.3

Q11 is Q10 with two numbers changed

This is the payoff for naming things. Question 11 asks for 55.53 kg in 2 hours 9 minutes 30 seconds, in kilograms per hour. The structure is identical, and because the answer is wanted in kilograms this time, the mass needs no conversion at all.

sheet-01/q10 → sheet-01/q11

```
  double sales, Time, average;

- sales   = 600e3;                    // grams
+ sales   = 55.53;                    // kilograms
- Time    = 15 + 30.0/60 + 30.0/3600;  // hours
+ Time    = 2 + 9.0/60 + 30.0/3600;    // hours
  average = sales / Time;

- cout << "average = " << average << " grams/hr" << endl;
+ cout << "average = " << average << " kg/hr" << endl;
```

OUTPUT

```
average = 25.7282 kg/hr
```

WHAT COULD FOOL YOU

- **Reading `=` as equals.** `average = sales / Time;` is an instruction, not a statement of fact. Writing `sales / Time = average;` is not C++.
- **`Time` and `time` are different names.** C++ is case sensitive throughout. The lecture's solutions use a capital `T`, so this guide does too; pick one and be consistent.
- **Using a variable before assigning one.** Declaring `double x;` and printing `x` gives whatever rubbish was in that memory. It may look like 0 on one run and not on the next.

- **Declaring inside the wrong braces.** A name declared inside `{ }` stops existing at the closing brace.
- **Forgetting the units.** Q10 wants grams per hour from a mass in kilograms; Q11 wants kilograms per hour from a mass in kilograms. The programs differ by a factor of 1000 and neither compiler will mention it.

CHECK YOURSELF

A classmate declares `int Time;` and assigns `Time = 15 + 30.0/60 + 30.0/3600;`. What does the program print for the average?

A 38688.9, because both divisions are now real

B 40000, the same wrong answer as before

C A value based on Time being rounded up to 16

Back to the sheet: **Q10, Q11 and Q12** are all written this way. Q12 has six quantities, which is exactly when naming them stops being optional.

CHAPTER 6

How many digits does cout show?

After this chapter you can make a number print in the form the question asks for, and you can explain why that never changes the value the program is holding.

Write › Compile › Evaluate › **Print**

6.1

Six significant digits, then a decision

You never asked for 38688.9 to have one decimal place. That is cout's default: show **six significant digits** - six digits counted from the first non-zero one, wherever the decimal point happens to fall. 38688.876947 has five digits before the point, so one is left for after it.

When six significant digits cannot describe the size of the number, cout switches to scientific notation rather than print something misleading. Question 12 walks straight into this.

what Q12 prints if you do nothing

```
double initialPopulation = 118058996;
cout << initialPopulation << endl;
```

OUTPUT

```
1.18059e+08
```

That is 1.18059×10^8, and it is not wrong - it is 118058996 rounded to six significant digits, which is all cout was asked for. It is simply useless as a population figure. Question 12 wants whole people.

6.2

Two switches, both from `<iomanip>`

`fixed` says "never use scientific notation". `setprecision(n)` sets the number of digits - and what it counts **depends on whether `fixed` is in effect**. That single fact causes most of the confusion around these two.

Both live in a second library, so a program that formats output needs two include lines. Move the dial below and read the sentence underneath it.

Notation

Digits asked for **`setprecision(6)`**

the line you would write

```
cout << setprecision(6) << value;
```

THE SAME THREE VALUES, PRINTED THREE WAYS

```
average   : 38688.9
people    : 1.18059e+08
increase  : 1.36693e+06
```

Without fixed, setprecision(6) means 6 significant digits in total, counted from the first non-zero digit. cout switches to scientific notation when the number is too big to show at that precision, which is why the population prints as 1.18059e+08.

Nothing above changes what the program is holding. All three values are the full-precision doubles throughout; only the characters sent to the console differ.

KEY IDEA

With `fixed`, `setprecision(n)` means *n digits after the decimal point*. Without it, `setprecision(n)` means *n significant digits in total*. Same call, different meaning, which is why Q12 uses both together.

6.3

Question 12, assembled

Births minus deaths gives 156.042 people per hour. Multiply by 24 and by 365 and a year adds 1366927.92 people. The population is a `double` because that yearly figure has a fraction in it, and truncating it to a whole number each year would lose people compounding over five years. It is printed as a whole number because half a person is not a sensible thing to report.

sheet-01/q12-population-forecast.cpp · the parts that matter

```cpp
#include <iostream>
#include <iomanip>
using namespace std;

double initialPopulation = 118058996;
double birthPerHr        = 226.625;
double deathPerHr        =  70.583;
double days              = 365;

double populationChangePerHr       = birthPerHr - deathPerHr;
double populationIncreaseInOneYear = days * 24 * populationChangePerHr;

cout << fixed << setprecision(0);
cout << "At the end of 2026 = "
     << initialPopulation + populationIncreaseInOneYear * 1 << endl;
```

OUTPUT

```
At the end of 2025 = 118058996
At the end of 2026 = 119425924
At the end of 2027 = 120792852
At the end of 2028 = 122159780
At the end of 2029 = 123526708
At the end of 2030 = 124893636
```

✓ CHECKED BY COMPILING IT

The figure for 2026 is 119425923.92 before printing. `fixed` with `setprecision(0)` rounds that to 119425924 on the way to the console, while the value inside the program keeps its .92 and carries it into the next year's arithmetic. The six lines above came from running the file in the course folder on 21 September 2026.

بالمصري

`setprecision` بيغيّر الشكل اللي بيتطبع على الشاشة بس. الرقم المحفوظ جوه البرنامج ما بيتغيّرش ولا بيتقرّب.

وخلي بالك: الإعداد ده بيفضل شغال على كل اللي بعده لحد ما تغيّره تاني.

WHAT COULD FOOL YOU

- **They are sticky.** `fixed` and `setprecision` stay in force for everything printed afterwards, not just the next item. Set them once, near the top, and remember you did.
- **Printing is not storing.** `setprecision(0)` does not round the variable. Print a value, then use it in a later calculation, and the calculation uses the full value - as Q12's compounding relies on.
- **Forgetting the second include.** `setprecision` without `#include <iomanip>` fails to compile. `fixed` happens to be available through `<iostream>`, so a program can compile with `fixed` alone and then break the moment you add precision.
- **Reading `e+08` as an error.** It is a correct number in scientific notation. The program is fine; only the presentation needs fixing.
- **Assuming six digits is a lot.** For a population of 118 million it is not even enough to reach the thousands column.

CHECK YOURSELF

After `cout << fixed << setprecision(0);`, a program prints the 2026 population and then computes 2027 from it. What is 2027 calculated from?

A 119425924, the rounded figure that was displayed

B 119425923.92, the full value the variable still holds

C 119425923, the value truncated to a whole number

Back to the sheet: only **Q12** needs `<iomanip>`. Everywhere else the default six digits is what the reference answers show, so do not add formatting the question did not ask for.

CHAPTER 7

The sheet, question by question

After this chapter you can finish all twelve questions, check each answer against a known-good output, and say which chapter every question is testing.

Write › Compile › Evaluate › Print

7.1

What each question is really asking

Twelve questions, five ideas. The right-hand column is what the finished program prints, produced by compiling the solution files in the course folder rather than worked out by hand. If your program prints something else, the chapter named beside it is where the answer is.

Q	What it is testing	Where	Output
1	One `cout` per line	Ch 1	`four lines of course information`
2	Repeating a statement	Ch 1	`Welcome to C++ ×5`
3	Spaces inside a string are data	Ch 2	`5 lines, 20 columns each`
4	`\t` and tab stops	Ch 2	`P / P*5 / P*10`
5	Brackets around a fraction	Ch 3	`12.1978`
6	cout receives one number, not ten	Ch 3	`100`
7	Integer division, on purpose	Ch 4	`3.01707  then  4`
8	No power operator	Ch 3	`36.6491 · 27.6`
9	Mixing int and double safely	Ch 3	`27.04 · 20.8`
10	Unit conversion, and the `.0`	Ch 4, 5	`38688.9 grams/hr`

Q	What it is testing	Where	Output
11	The same shape, different units	Ch 4, 5	`25.7282 kg/hr`
12	`double` plus output formatting	Ch 5, 6	`118058996 → 124893636`

7.2

Where things stand

Two questions have a reference answer that does not match the wording on the sheet. Neither is a trap and neither is your fault; they are listed here so that you recognise the discrepancy instead of assuming your program is broken.

SETTLED

Everything in chapters 1 to 6 is language behaviour, not opinion

Integer division truncating, cout showing six significant digits by default, `*` binding tighter than `+`, `setprecision` changing only the output: these are defined by the C++ standard and behave identically in Dev-C++ on the lab machines and clang++ on a Mac. Every output on this page was produced by compiling and running the code.

OPEN · Q4

Column headings

The sheet asks for `P`, `P*5`, `P*10`. The reference screenshot heads them `p^2`, `p^3` over the same numbers. **What it changes:** three words in one string. Follow the sheet unless your tutor says otherwise.

OPEN · Q7

How many terms

The sheet's formula ends at `-1/15`; the reference screenshot stops at `+1/13`. **What it changes:** 3.01707 versus 3.28374. Both files are in the folder, and the discussion the question asks for is the same either way.

OPEN · Q10

The unit label

The reference screenshot prints `grames/hr`. **What it changes:** one letter of output text, no arithmetic. The solution file in the folder spells it `grams/hr`.

7.3

Before you hand it in

Ticks are kept in this browser only, so they will not follow you to another machine.

- [ ] **Every file compiles with no errors**

Not "it ran last time". Compile each one again after your last edit.

- [ ] **Every division checked separately**
  Look at each `/` on its own. Are both sides whole numbers? Did you mean that?

- [ ] **No answer is suspiciously round**
  40000 and 4 are the shape integer division produces. If a messy question gave a tidy answer, check it.

- [ ] **Units match what was asked**
  Q10 wants grams per hour from kilograms; Q11 wants kilograms per hour. A factor of 1000 separates them.

- [ ] **Q3 still has its trailing spaces**
  All five strings are 20 characters. An editor that trims whitespace on save will have shortened them.

- [ ] **You gave C++ the question, not the answer**
  Q5, Q6 and Q8 should contain the expression, not a number you worked out elsewhere.

- [ ] **Output compared against the table in 7.1**
  Character for character, including how many digits appear.

- [ ] **Each file starts with a comment naming the question**
  Matching the lecture's `// Sheet 01, Q_01` style. It is how your work gets marked.

+ **If you want to go further this week** · optional, not on the sheet

Sources and checks

- **Sheet (1)**, Computer Languages & Applications, Prof. Mahmoud M. Khater. All twelve questions, read 21 September 2026. Kept in the course folder at `course-materials/sheets/sheet-01.pdf`.
- **Lecture GEN241_01, Introduction**, same author, 7 pages, read 21 September 2026. Examples 1 to 3 on pages 4 to 7 are the source for chapters 1 and 3; the lecture's own printed result for Example 3,

`0.39759`, was reproduced by compiling it.

- **Reference solutions**: twelve screenshots supplied with the sheet, stored at `course-materials/sheet-01-reference-solutions/`. Three disagreements with the sheet's wording are listed on the board in chapter 7 and in `sheet-01/REVIEW.md`; everything else matches.
- **Every number on this page was produced by compiling and running code**, not by hand and not copied from the screenshots. Toolchain: Apple clang 21.0.0 for arm64, `-std=c++17 -Wall -Wextra`, macOS 25.6, 21 September 2026. All twenty-one programs in the course folder compile with no warnings. The figures were then re-derived independently in Python and agreed to every digit shown.
- **The stall** (600 kg in 15 h 30 min 30 s) is question 10 of the sheet, used unchanged as this guide's running example. The population figures in chapter 6 are the sheet's own stated assumptions, not a demographic forecast.

Written for first-year Electrical Power and Machines Engineering students taking GEN 241, and for the teaching assistant running the week 1 lab. It covers Sheet 1 only: no loops, no conditions, no input. The widgets reproduce C++'s arithmetic and output rules for the small expressions on this sheet; they are a teaching model, not a compiler, so when a widget and your compiler disagree, your compiler is right.